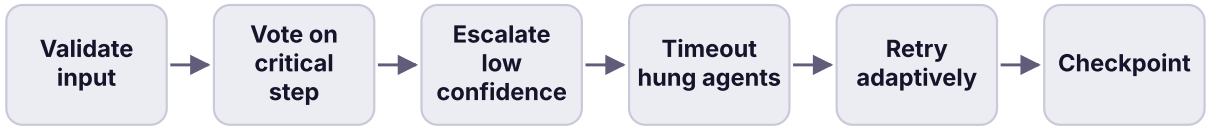
Robustness & Fault Tolerance Patterns

1. Robustness is a spectrum

No single pattern makes a system robust. You **chain** fault-tolerance patterns to reach the reliability the use case needs, and measure it: success rate under fault injection, mean time to recovery, graceful-degradation coverage.

✎ DIAGRAM



A loan application might chain all of these.

2. Reliability through redundancy

- **Parallel Execution Consensus**: run the critical task on several agents and require agreement before proceeding
- **Majority Voting Across Agents**: take the majority answer of N agents

> ⚠ Voting multiplies cost by N and only helps when failures are independent. Use it on critical, hard-to-reverse decisions (a loan approval), not every step.

3. Keeping a run alive

- **Watchdog Timeout Supervisor**: kills or restarts any agent past its deadline, so one stuck agent can't freeze the system
- **Delayed Escalation Strategy**: on low confidence, try lighter recovery first, then a stronger model or a human
- **Adaptive Retry with Prompt Mutation**: analyse why it failed and change the approach before retrying, instead of repeating the same failing call
- **Auto-Healing Agent Resuscitation**: detect a crashed agent and restart it with its state
- **Incremental Checkpointing**: save state after each stage so a long pipeline resumes from the last good point

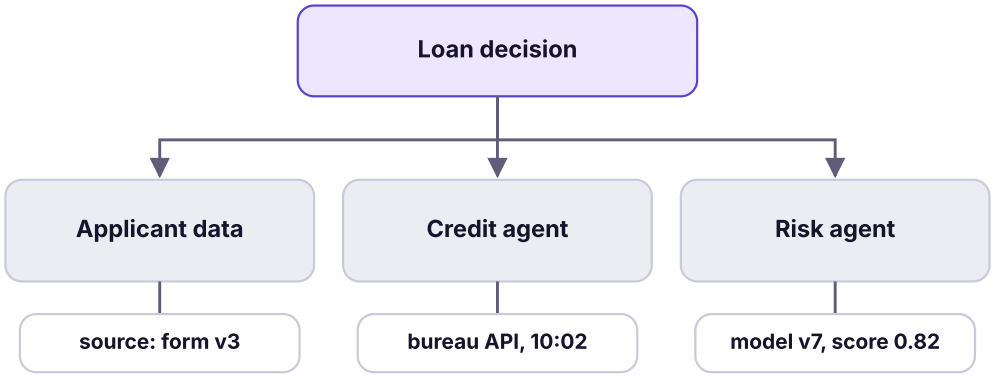
4. Security patterns

- **Agent Self-Defense**: the agent spots and neutralises adversarial input aimed at it (e.g. injection in feedback it summarises)
- **Agent Mesh Defense**: defence in depth; a compromised agent is contained by its peers and can't reach sensitive tools or data
- **Execution Envelope Isolation**: run risky actions such as code execution in a sandbox

5. Graceful degradation and auditability

- **Rate-Limited Invocation** (respect downstream limits) · **Fallback Model Invocation** (fail over to a secondary model) · **Trust Decay and Scoring** (down-weight flaky agents over time) · **Canary Agent Testing** (upgrade a small slice first) · **Optimising Translation Overhead** (summarise once, don't re-pass large documents)
- **Causal Dependency Graph**: records which inputs, agents and steps caused each output, so any decision can be traced afterwards. The observability backbone AgentOps needs

✎ DIAGRAM



A causal dependency graph lets you answer 'why was this loan rejected?' after the fact.

SAY IT

"Robustness comes from chaining patterns: validate inputs, vote on the critical decision, escalate low confidence, time out hung agents, retry adaptively, and checkpoint progress."